Introduction to Applied Machine Learning

09: Transformer Language Models and Autoregressive Generation

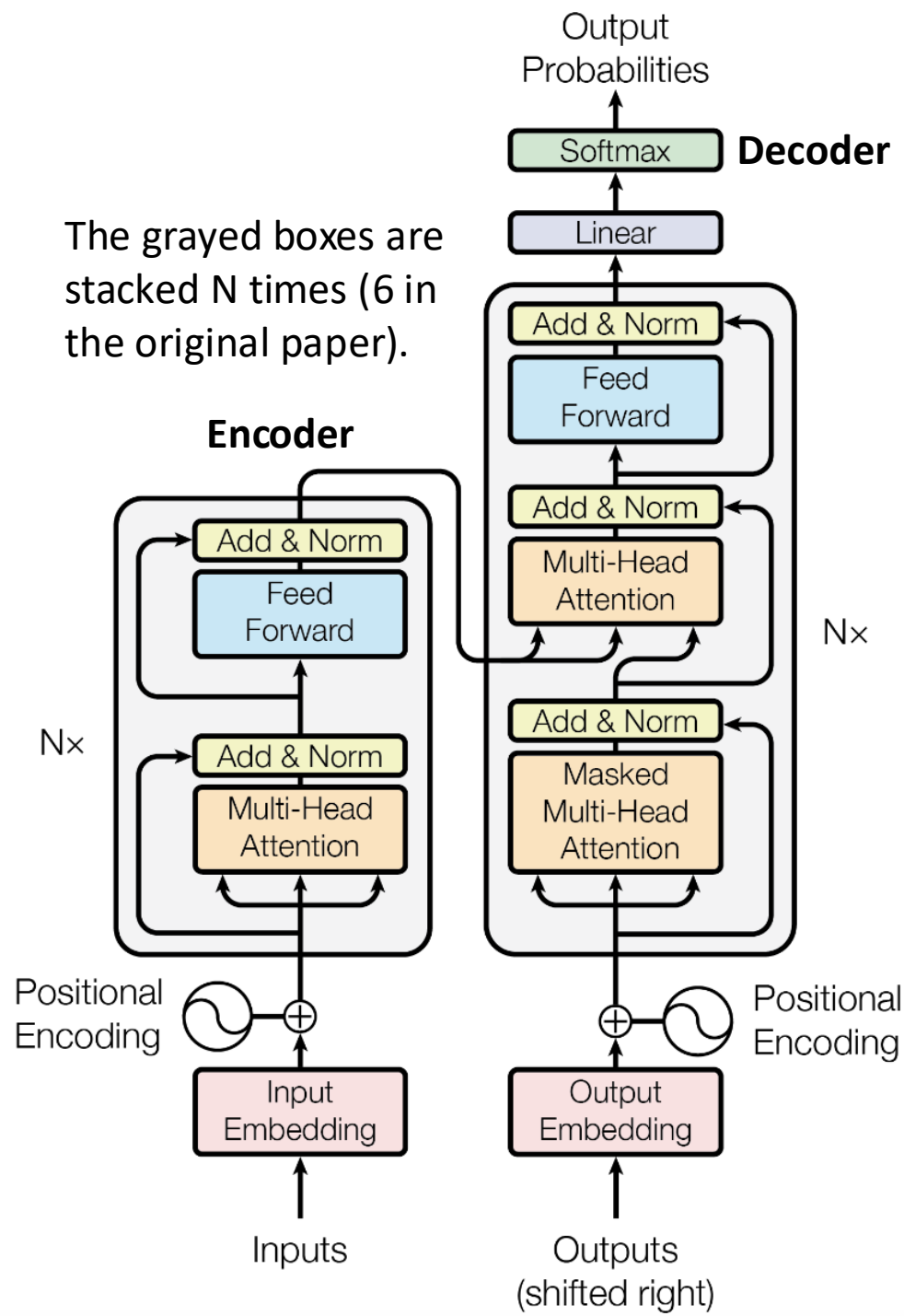
🎓 Machine Learning Adventure 🚀

1. 🤖 Introduction to Machine Learning
2. 🧠 Artificial Neural Networks and Computer Vision
3. 🔤 Transformers and Language Models
4. 🎨 GANs and Diffusion for Image Generation
5. 🎮 Reinforcement Learning and Behavior
6. 🏆 Final Projects

Today's Plan

1. **Input**:
   - Tokenization
   - Word Embedding
   - Positional Encoding
2. **Encoder** (BERT style):
   - Pretraining
   - Contextual Embeddings
   - Sequence Classification
3. **Decoder** (GPT style):
   - Autoregressive Generation
   - Causal Masking and Pretraining

The grayed boxes are stacked N times (6 in the original paper).



Input: Tokenization and Embeddings

Tokenization

```
sentence = "The quick fox, look how it runs willingly!"

from transformers import GPT2TokenizerFast
tokenizer = GPT2TokenizerFast.from_pretrained('gpt2')

encoded = tokenizer.encode(sentence)
print(encoded)
```

[464, 2068, 21831, 11, 804, 703, 340, 4539, 30981, 0]

is a list of **token indices** in the **vocabulary**.

`tokenizer.vocab['The']`	464
`tokenizer.vocab['!']`	0
`print(tokenizer.vocab_size)`	50257

`tokenizer.decode(encoded)` 'The quick fox, look how it runs willingly!'

Word Embeddings

tokenizer.encode('the black cat naps')

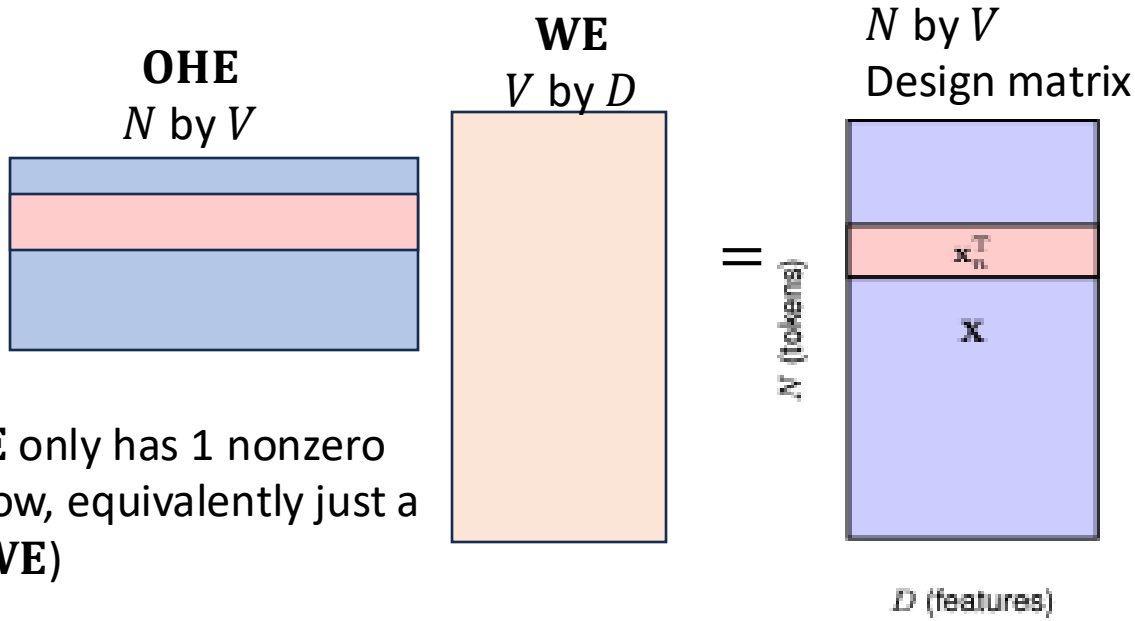
[1169, 2042, 3797, 299, 1686]

One hot encoding matrix: row per token.

Sequence length = # Tokens N

[[0, 0, …, 0, 1, 0, …, 0, 0],
[0, 0, 0, …, 0, 1, …, 0, 0],
[0, 0, 0, 0, …, 0, 1, …, 0],
[0, …, 1, 0, …, 0, 0, 0, 0],
[0, …, 0, 1, 0, …, 0, 0, 0]]

Map **OHE** to the model input matrix **X** by multiplying by a word embedding matrix **WE**.



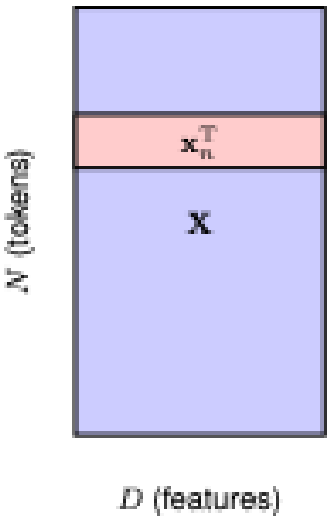
(Since **OHE** only has 1 nonzero entry per row, equivalently just a lookup in **WE**)

Order Matters

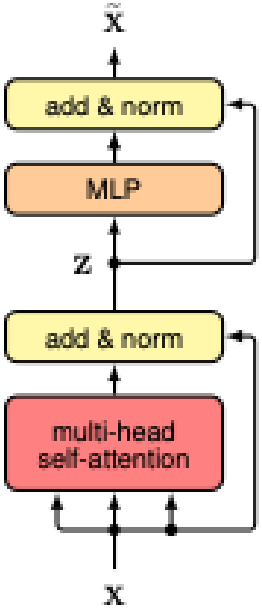
Consider the inputs:

a) The food was good, not bad at all

b) The food was bad, not good at all



But a transformer block would give the **same** transformation just with the rows permuted.



We explicitly add a ***positional encoding/embedding***. Without this a transformer could not distinguish

Positional Encoding/Embedding Example

A single linear layer embeds each position index into a vector of length D (the embedding dimension). Add this to the word embedding for the token at that position, also of length D.

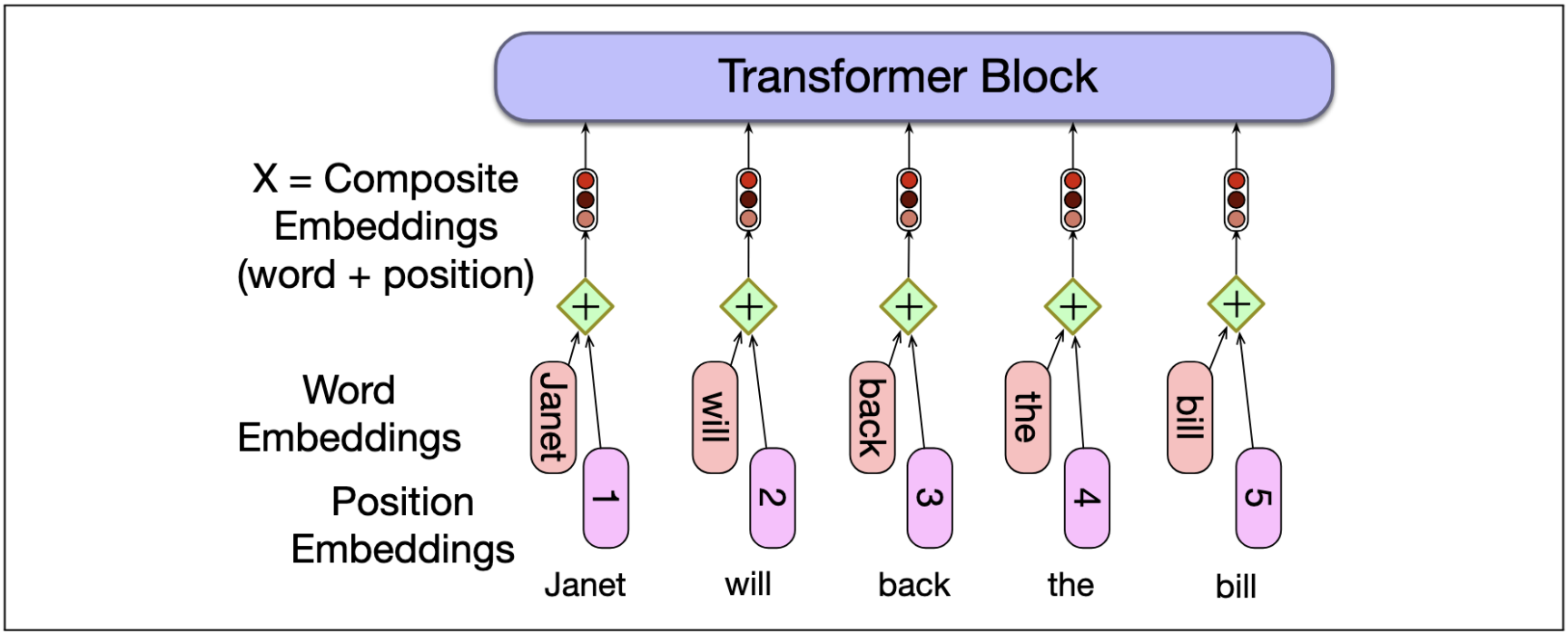


Figure 9.13 A simple way to model position: add an embedding of the absolute position to the token embedding to produce a new embedding of the same dimensionality.

Transformer Input Embeddings in PyTorch

Embedding

CLASS torch.nn.Embedding(*num_embeddings*, *embedding_dim*, *padding_idx=None*, *max_norm=None*, *norm_type=2.0*, *scale_grad_by_freq=False*, *sparse=False*, *_weight=None*, *_freeze=False*, *device=None*, *dtype=None*) [SOURCE]

A simple lookup table that stores embeddings of a fixed dictionary and size.

This module is often used to store word embeddings and retrieve them using indices. The input to the module is a list of indices, and the output is the corresponding word embeddings.

Parameters

In: vocab size for word embedding, max sequence length or **context window** for positional embedding.

- **num_embeddings** (*int*) – size of the dictionary of embeddings
- **embedding_dim** (*int*) – the size of each embedding vector

Out: Model embedding dimension

Input Embeddings in PyTorch

```python
import torch
from torch import nn

# just examples
vocab_size = 8
max_seq_len = 20
embed_dim = 4

# tensor of vocabulary indices from tokenizer
T = torch.tensor([3, 7, 4, 2, 1] )

word_embed = nn.Embedding(vocab_size, embed_dim)
pos_embed = nn.Embedding(max_seq_len, embed_dim)

# converting input to embedding space
X = word_embed(T) + pos_embed(torch.arange(len(T)))
print(X)
```

✓ 0.0s

```
tensor([[ 0.0646, -0.4649, -1.4847,  0.7210],
        [-1.5289, -1.1245,  0.1922,  1.9141],
        [ 2.5338,  1.7828,  2.6512,  0.5155],
        [ 1.4380,  0.3687, -1.0723,  2.3418],
        [ 0.1349,  0.3816, -0.5108, -0.4701]], grad_fn=<AddBackward0>)
```

```python
torch.arange(5)
```

✓ 0.0s

```
tensor([0, 1, 2, 3, 4])
```

Encoder Architectures

Sequence Classification with Encoder-Only Transformers

Recall the toxic speech detection assignment:

"I hate ___ people, they are so awful"

Toxic?

"It is so awful when people hate on ___ people"

Not toxic? Even though uses roughly the same words?

How can we use an Encoder-only Transformer to classify natural language text?

Encoder-Only Bidirectional Transformer Language Models

Encoders, such as the Google BERT model, allow all-pairs bidirectional self-attention.

This is the default behavior of the attention mechanism, different from what we will later see for text generation.

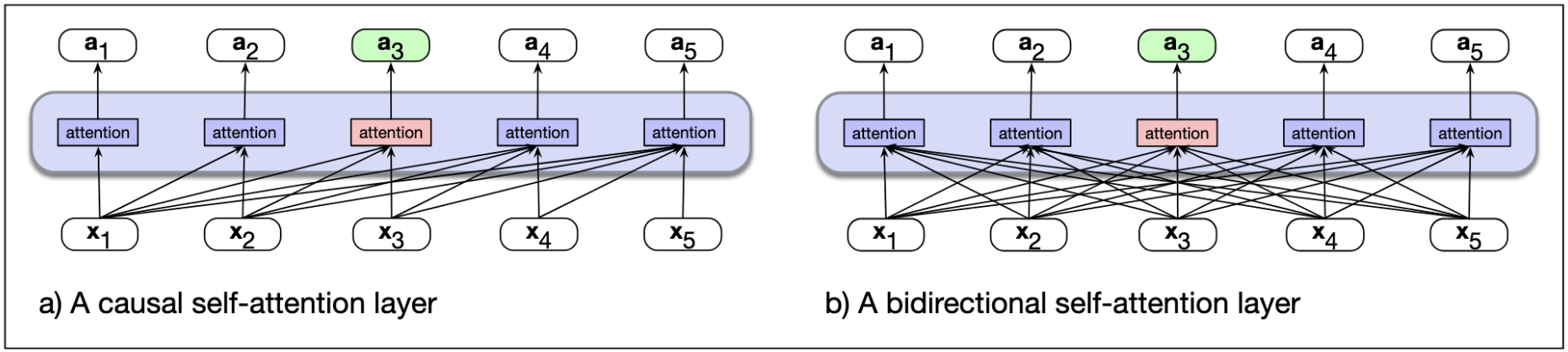


Figure 11.1 (a) The causal transformer from Chapter 9, highlighting the attention computation at token 3. The attention value at each token is computed using only information seen earlier in the context. (b) Information flow in a bidirectional attention model. In processing each token, the model attends to all inputs, both before and after the current one. So attention for token 3 can draw on information from following tokens.

Pretraining Encoder-Only Bidirectional Transformers

- Encoder are often **pretrained** to generate useful representations of text,
- And can later be **fine-tuned** for specific tasks like sequence classification.

Use a **cloze** style fill-in-the-blank task:

`The ____ of Walden Pond is so beautiful.`

Where we **mask** out the missing word in the input.

Self-Supervised Pretraining of Encoder-Only Transformers

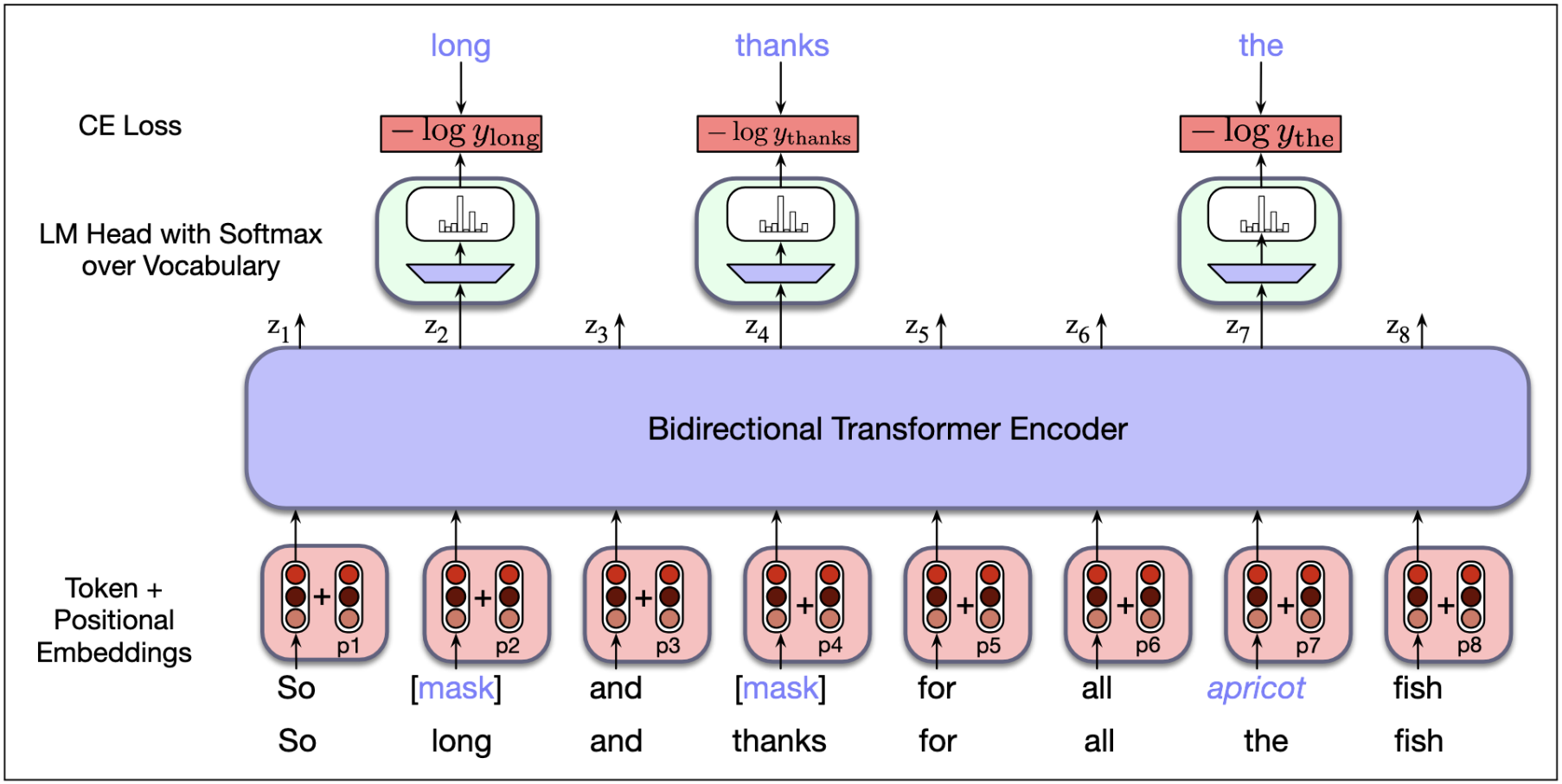
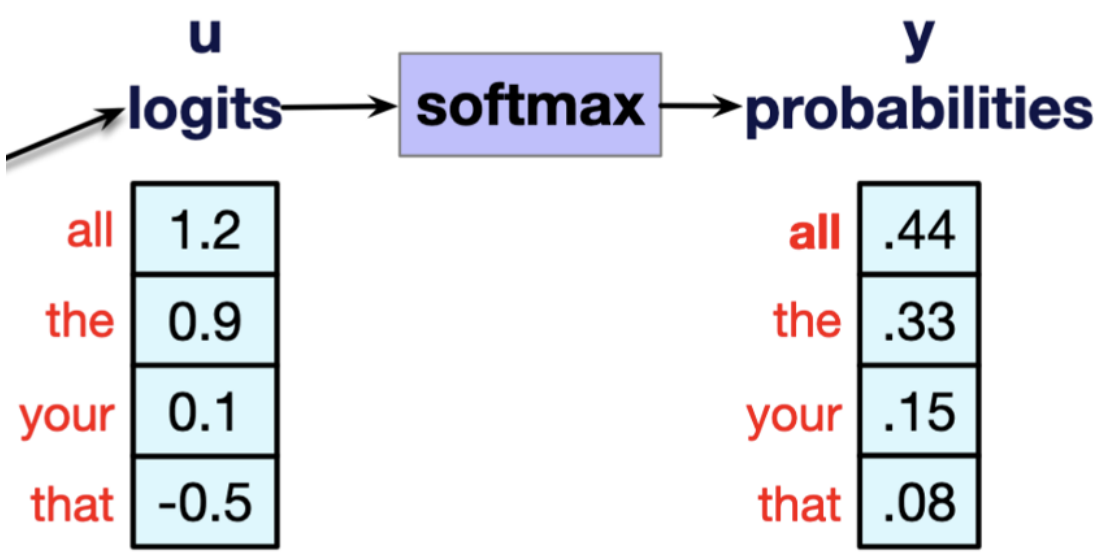


Figure 11.3 Masked language model training. In this example, three of the input tokens are selected, two of which are masked and the third is replaced with an unrelated word. The probabilities assigned by the model to these three items are used as the training loss. The other 5 tokens don't play a role in training loss.

Language Modeling Head

- A standard multiclassification output over the entire vocabulary.
- “unembedding layer” is a fully connected linear layer from the output of the last Transformer block to the vocabulary size number of output units.



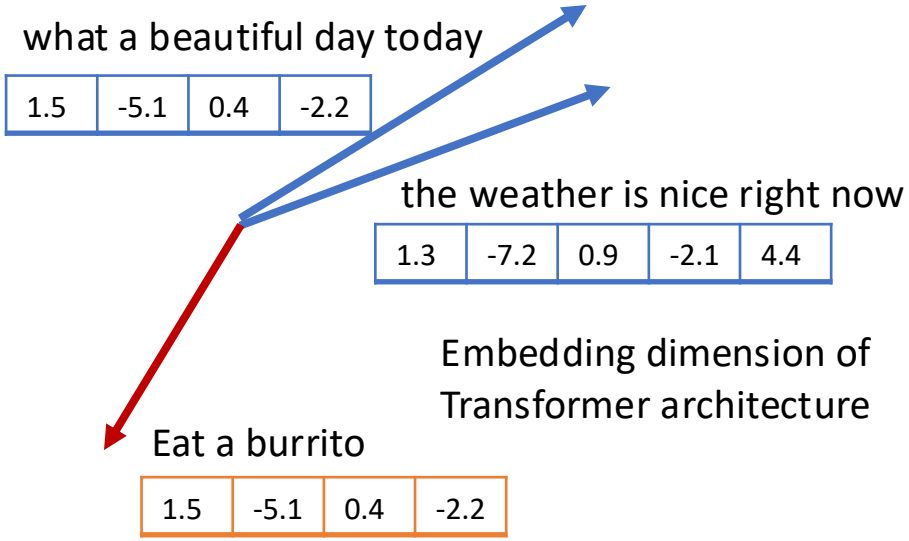
Contextual Embeddings

Word Embedding

- A word embedding is a numerical vector that represents the semantic meaning of a word.
- Semantically similar words are close in embedding space.

Contextual Embedding

- Encoder language models compute a **contextual embedding,** a numerical vector capturing the semantic meaning of a word in context.
- BERT includes a CLS embedding for the whole sequence.



Sequence Classification with Encoder-Only Bidirectional Transformers

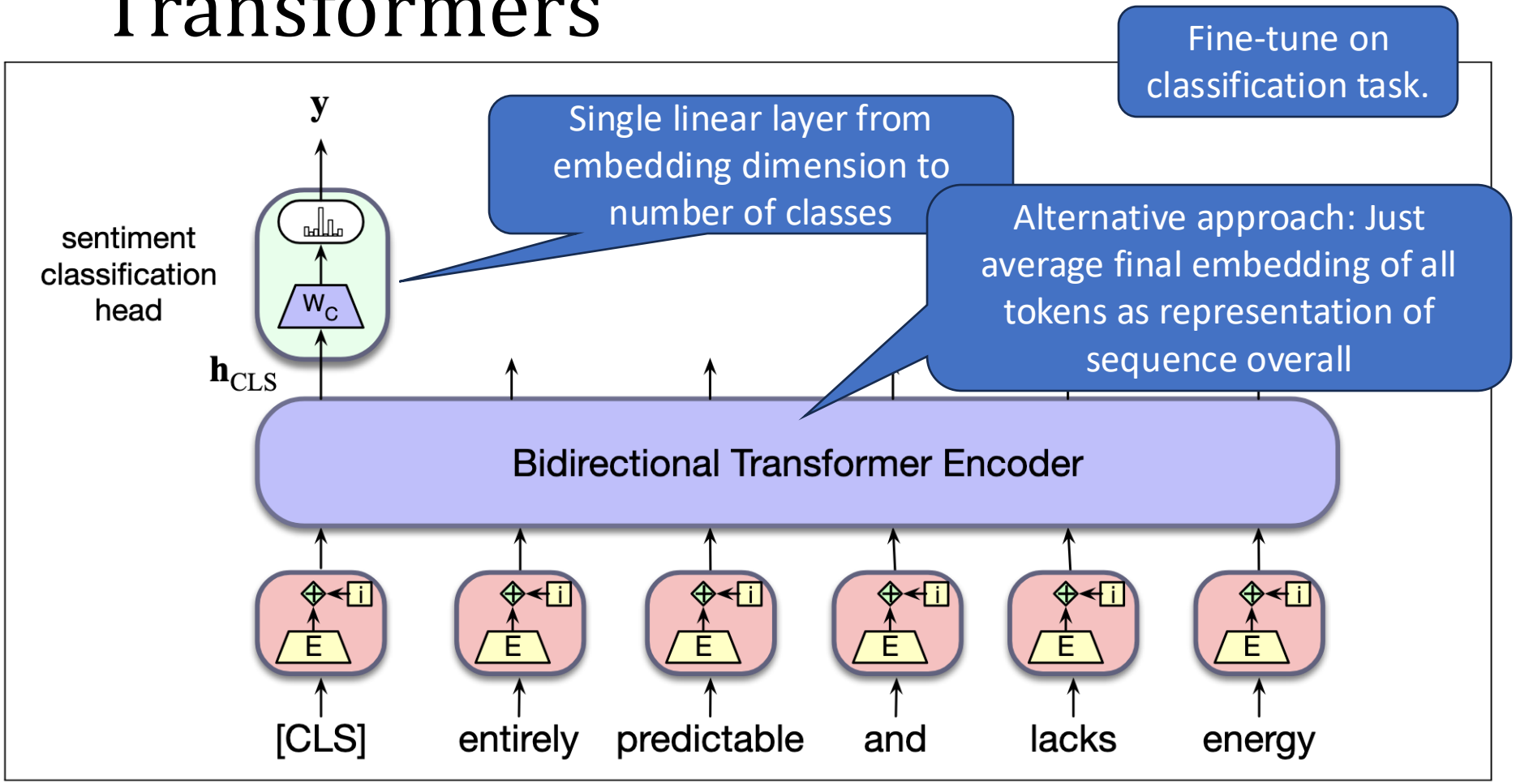


Figure 11.9 Sequence classification with a bidirectional transformer encoder. The output vector for the [CLS] token serves as input to a simple classifier.

Decoder Architectures and Autoregressive Generation

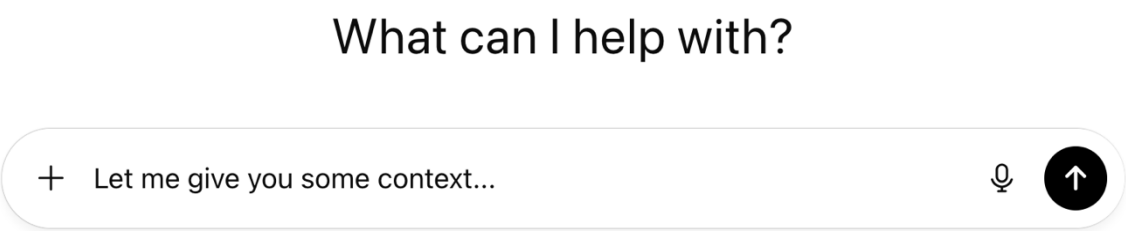
Language Models and Conditional Probability

A statistical model of language can be represented by the conditional probability of the next token, given all the previous ones, since

$$\Pr(\boldsymbol{x}_1, \dots, \boldsymbol{x}_N) = \prod_{i=1}^{N} \Pr(\boldsymbol{x}_i | \boldsymbol{x}_1, \dots, \boldsymbol{x}_{i-1})$$

A model that predicts $\Pr(\boldsymbol{x}_i | \boldsymbol{x}_1, \dots, \boldsymbol{x}_{i-1})$ is a **causal** language model (Example: GPT series models.)

$\boldsymbol{x}_1, \dots, \boldsymbol{x}_{i-1}$ is the **context** of a given prediction



Inference Time: Getting Next Token when NOT TRAINING

Predicting $\Pr(\boldsymbol{w}_i|\boldsymbol{w}_1, \ldots, \boldsymbol{w}_{i-1})$

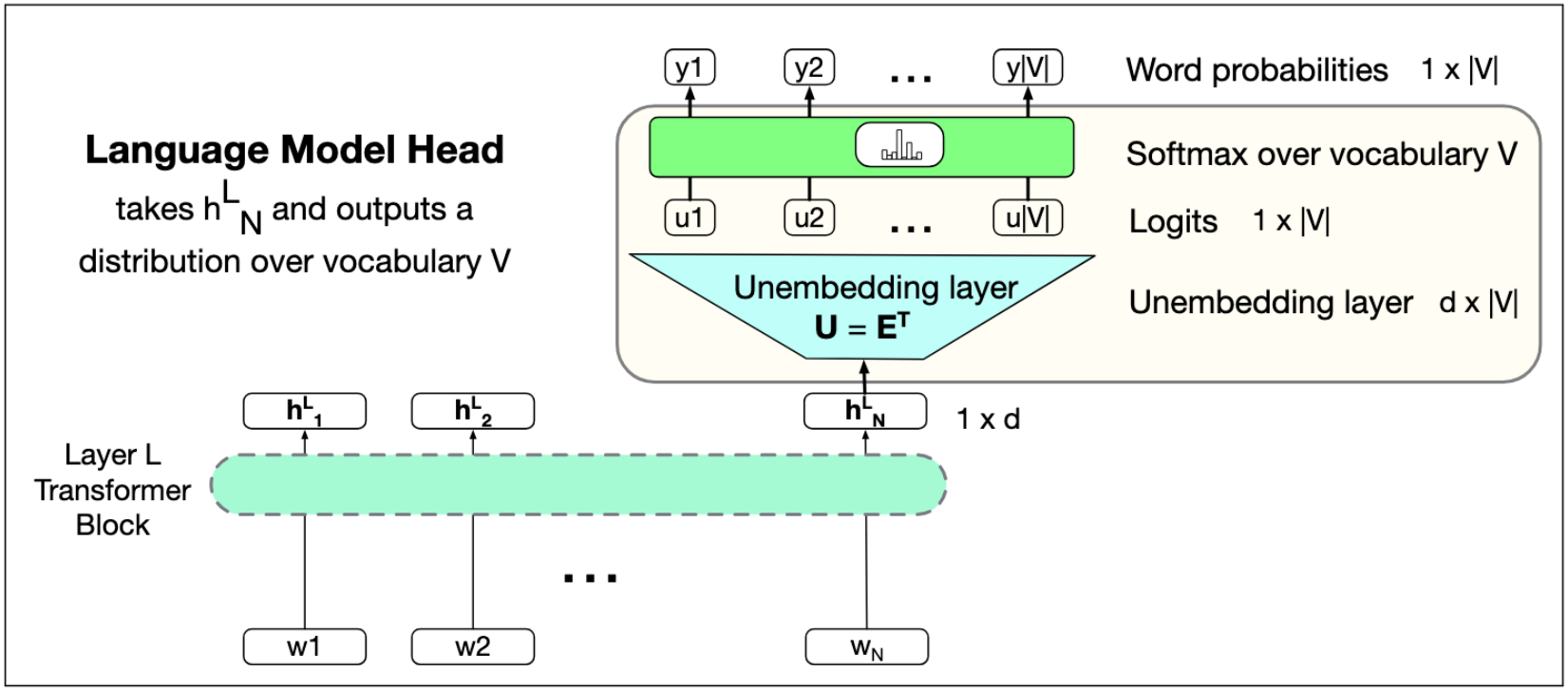


Figure 9.14 The language modeling head: the circuit at the top of a transformer that maps from the output embedding for token N from the last transformer layer ($\mathbf{h}_N^L$) to a probability distribution over words in the vocabulary V.

Note that you input the whole prior sequence, but only use the final representation of the **last token** to predict the next word.

Autoregressive Sequence Generation: Conceptual

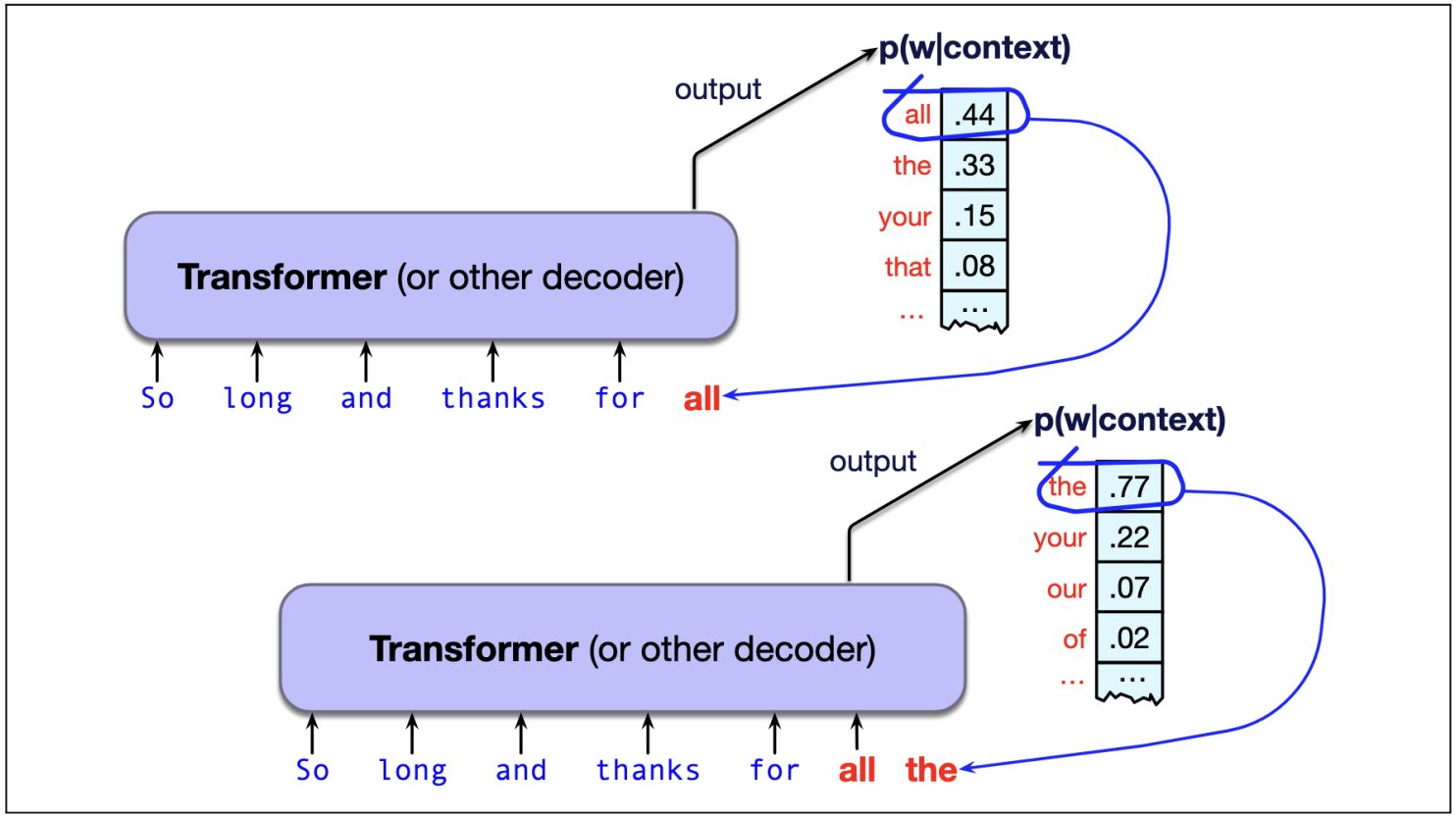


Figure 7.2 Turning a predictive model that gives a probability distribution over next words into a generative model by repeatedly sampling from the distribution. The result is a left-to-right (also called autoregressive) language models. As each token is generated, it gets added onto the context as a prefix for generating the next token.

Autoregressive Sequence Generation with Transformer

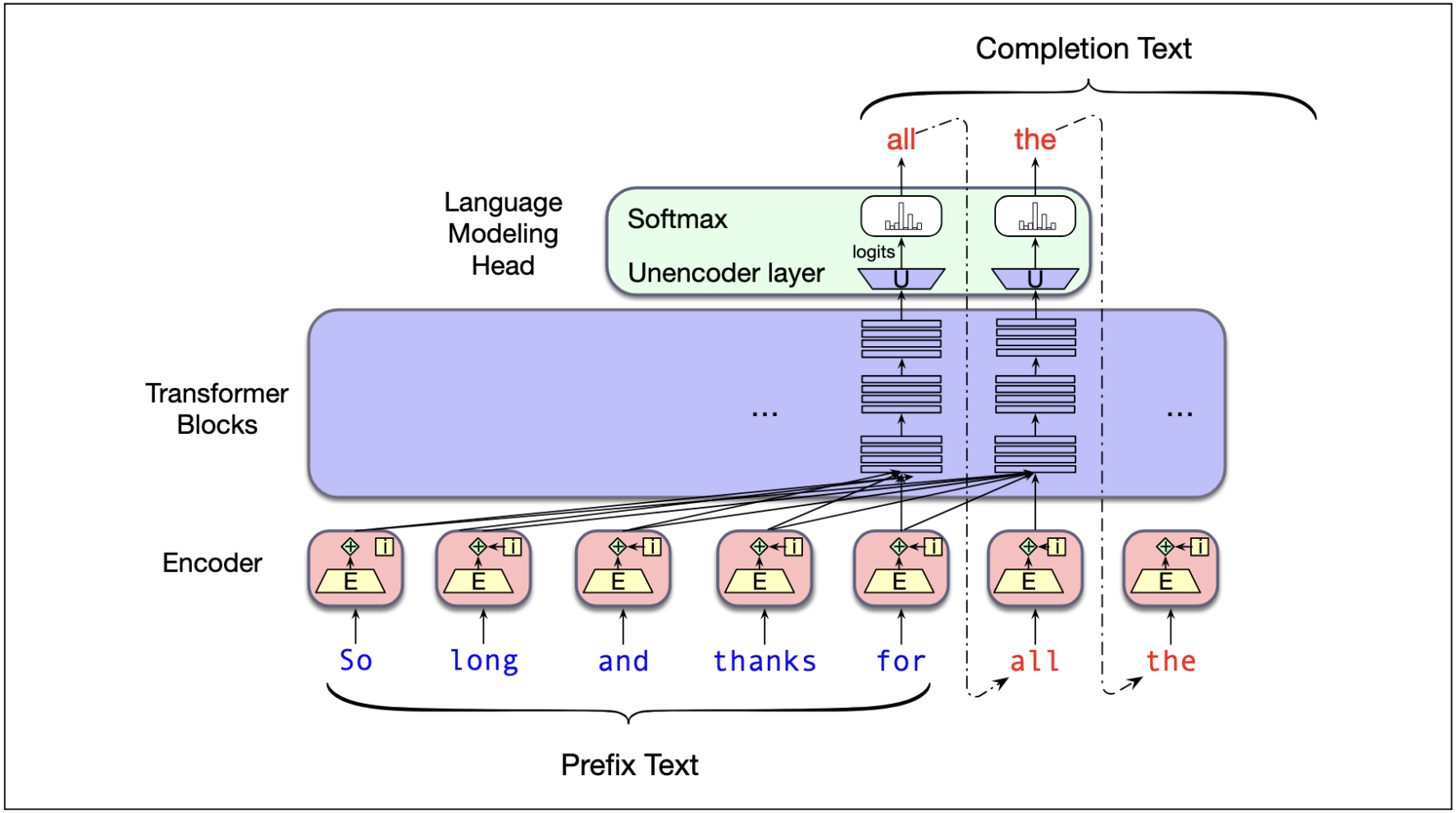


Figure 10.1 Left-to-right (also called autoregressive) text completion with transformer-based large language models. As each token is generated, it gets added onto the context as a prefix for generating the next token.

Stopping Criteria

Two approaches, often use both:

- max_new_tokens parameter: Simply hard-stop when you have generated a certain number of tokens.

- Early stopping: Add a special reserved EOS token in the vocabulary: Stop whenever this is generated.
  - During training, model learns to predict EOS at good stopping points.

Greedy Search → Sampling with Temperature

Given probabilities $y_1, y_2, \ldots, y_V$ for every token in the vocabulary, how to choose a token?

- **Greedy Search:** Choose $\mathrm{argmax}_j\, y_j$.
- **Sampling with Temperature.** We can also directly *sample* from the probability distribution generated by the softmax function.
  - Introduce a ***temperature*** scaling hyperparameter τ on the unnormalized logits $\boldsymbol{z}$ to control the variance.
  - Choose token i with probability to

$$\frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}}$$

Sampling and Temperature

Given unnormalized logits $\boldsymbol{z}$, choose token i with probability:

$$\frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}}$$

Greater τ → more entropy
Lesser τ → less entropy

softmax output with temperature τ

	logits	close to greedy τ=0.1	τ=0.5	normal softmax τ=1	τ=10	close to uniform τ=100
all	1.2	.95	.59	.44	.27	.25
the	0.9	.05	.32	.33	.26	.25
your	0.1	0	.07	.15	.24	.25
that	-0.5	0	.02	.08	.23	.25

low temperature sampling (towards greedy)

high temperature sampling (towards uniform)

Forward Propagation Complexity

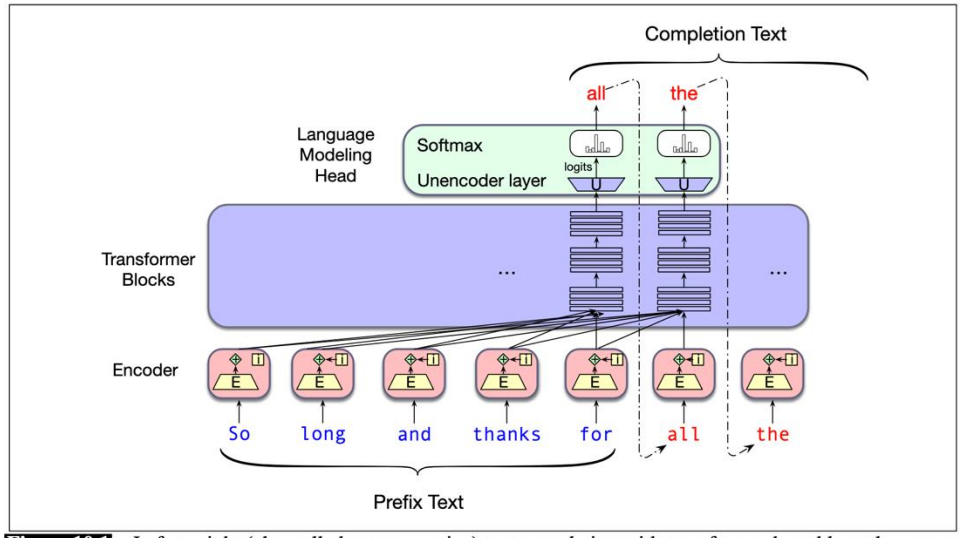


Figure 10.1 Left-to-right (also called autoregressive) text completion with transformer-based large language models. As each token is generated, it gets added onto the context as a prefix for generating the next token.

Input sequence length N
Embedding dimension D

Recall: transformer block has computational complexity $O(N^2D + ND^2)$.

Usually, a constant number of blocks in the overall Transformer.

Short-context ($N < D$)
Forward propagation complexity $O(ND^2)$.

Dominated by embedding dimension.

Long-context ($N > D$)
Forward propagation complexity $O(N^2D)$.

Quadratic dependence on the context.

Autoregressive Sequence Generation Complexity

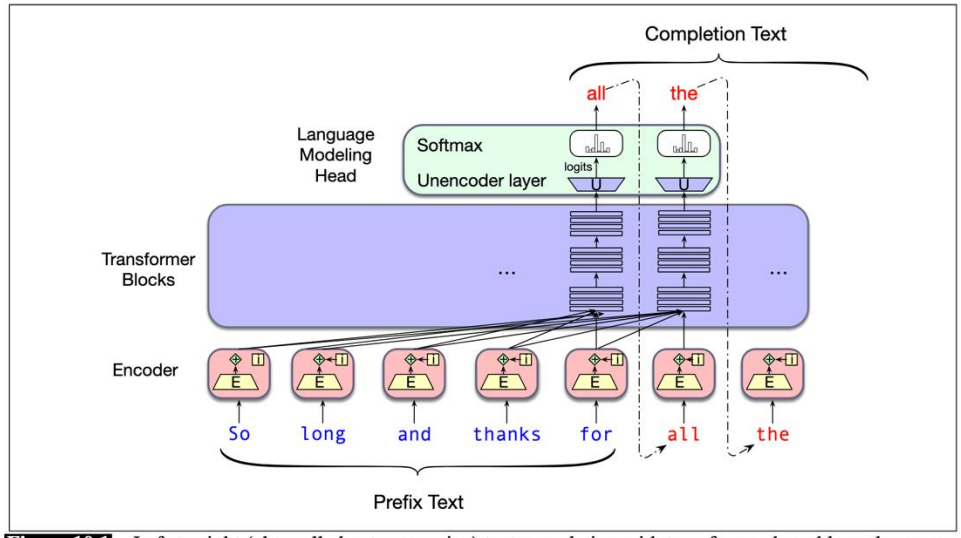


Figure 10.1 Left-to-right (also called autoregressive) text completion with transformer-based large language models. As each token is generated, it gets added onto the context as a prefix for generating the next token.

Input sequence length N
Embedding dimension D

Recall: transformer block has computational complexity $O(N^2D + ND^2)$.

To generate T new tokens from $N \gg D$ (**long-context regime**):

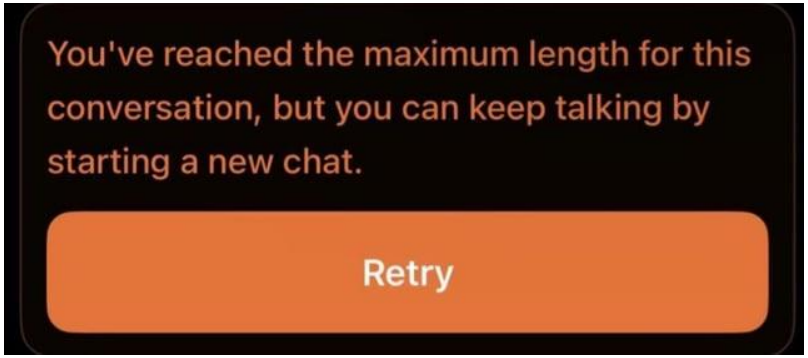
$$N^2D + (N+1)^2D + (N+2)^2D + \cdots + \big(N + (T-1)\big)^2 D$$

is $O(DT^3 + DT^2N + DN^2T)$

In the long-context regime, generation is dominated by the size of the (growing) context.

Context Window

- Most causal language models have a **context window**, a maximum number of input tokens they can process.
- Determined mostly by:
  - Size and type of of positional encoding, and
  - Quadratic scaling with respect to sequence length N



Self-Supervised Training Causal LM Decoder-Only Transformer

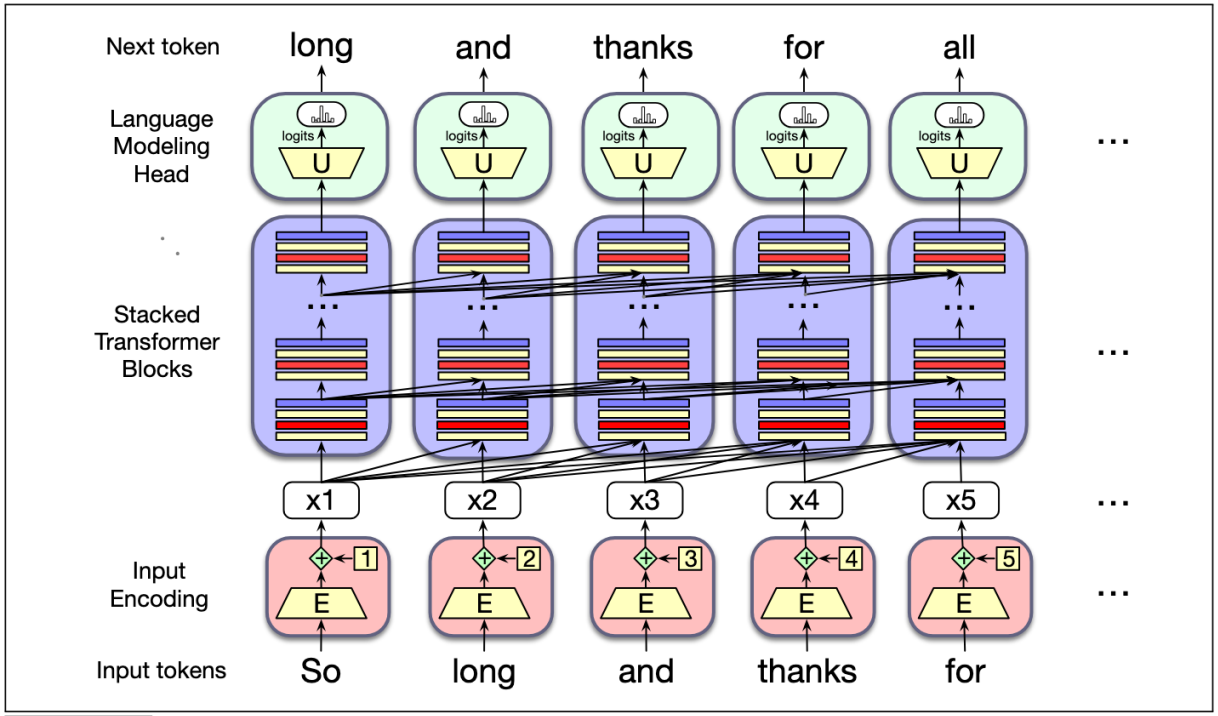
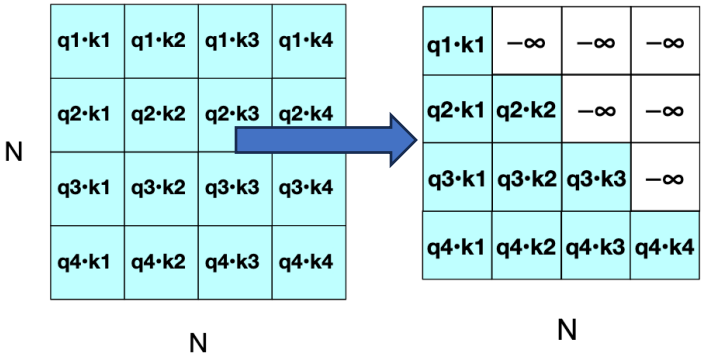


Figure 9.1 The architecture of a (left-to-right) transformer, showing how each input token get encoded, passed through a set of stacked transformer blocks, and then a language model head that predicts the next token.

Attention mechanisms need **causal masking** to prevent attending to later tokens during training (like the one you want to predict!).

Entire sequence in as input, **same sequence offset by one** as the predictive target. Average cross entropy loss at each prediction. Enables training on an entire sequence in parallel!

Making Input to Causal LM Decoder-Only Transformer



Upper triangular masking prevents token i from attending to later tokens $j > i$.

Can provide a Boolean mask to the forward method of the PyTorch nn.MultiheadAttention.

[documentation]

MultiheadAttention

CLASS torch.nn.MultiheadAttention(*embed_dim*, *num_heads*, *dropout=0.0*, *bias=True*, *add_bias_kv=False*, *add_zero_attn=False*, *kdim=None*, *vdim=None*, *batch_first=False*, *device=None*, *dtype=None*) [SOURCE]

Allows the model to jointly attend to information from different representation subspaces.

Method described in the paper: Attention Is All You Need.

Multi-Head Attention is defined as:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(head_1, \dots, head_h)W^O$$

where $head_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$.

forward(*query*, *key*, *value*, *key_padding_mask=None*, *need_weights=True*, *attn_mask=None*, *average_attn_weights=True*, *is_causal=False*) [SOURCE]

Compute attention outputs using query, key, and value embeddings.

attn_mask (*Optional*[*Tensor*]) – If specified, a 2D or 3D mask preventing attention to certain positions. Must be of shape (L, S) or $(N \cdot \text{num_heads}, L, S)$, where N is the batch size, L is the target sequence length, and S is the source sequence length. A 2D mask will be broadcasted across the batch while a 3D mask allows for a different mask for each entry in the batch. Binary and float masks are supported. For a binary mask, a `True` value indicates that the corresponding position is not allowed to attend. For a float mask, the mask values will be added to the attention weight. If

Computing Causal Mask

```python
seq_len = 5

true_matrix = torch.ones(size=(seq_len, seq_len), dtype=torch.bool)
print(true_matrix)

upper_triangle = torch.triu(true_matrix, diagonal=1)
print(upper_triangle)
```

✓ 0.0s

```
tensor([[True, True, True, True, True],
        [True, True, True, True, True],
        [True, True, True, True, True],
        [True, True, True, True, True],
        [True, True, True, True, True]])
tensor([[False,  True,  True,  True,  True],
        [False, False,  True,  True,  True],
        [False, False, False,  True,  True],
        [False, False, False, False,  True],
        [False, False, False, False, False]])
```

To omit the main upper left to lower right diagonal

Reminder: Only Need to Use the Last Output for Inference

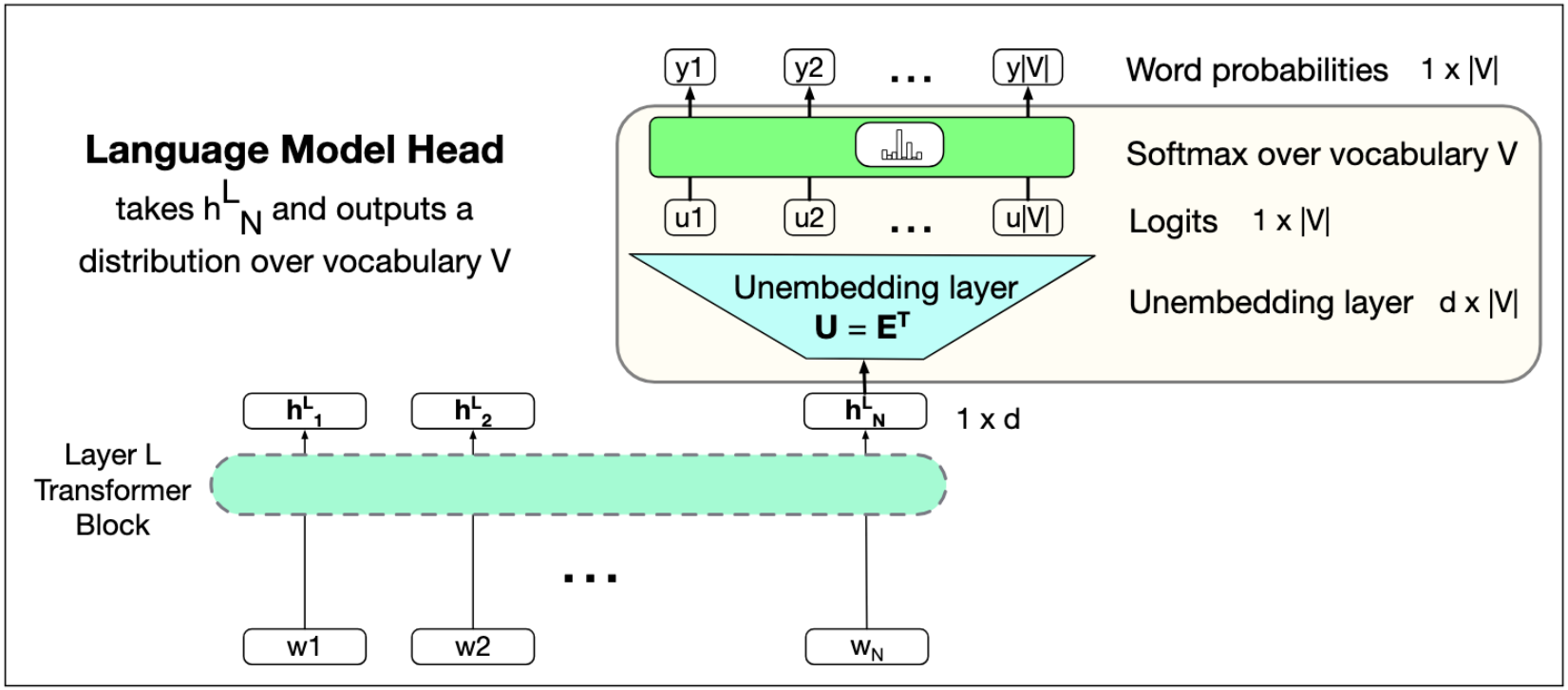


Figure 9.14 The language modeling head: the circuit at the top of a transformer that maps from the output embedding for token N from the last transformer layer ($\mathbf{h}_N^L$) to a probability distribution over words in the vocabulary V.

Optional: Encoder-Decoder Architectures

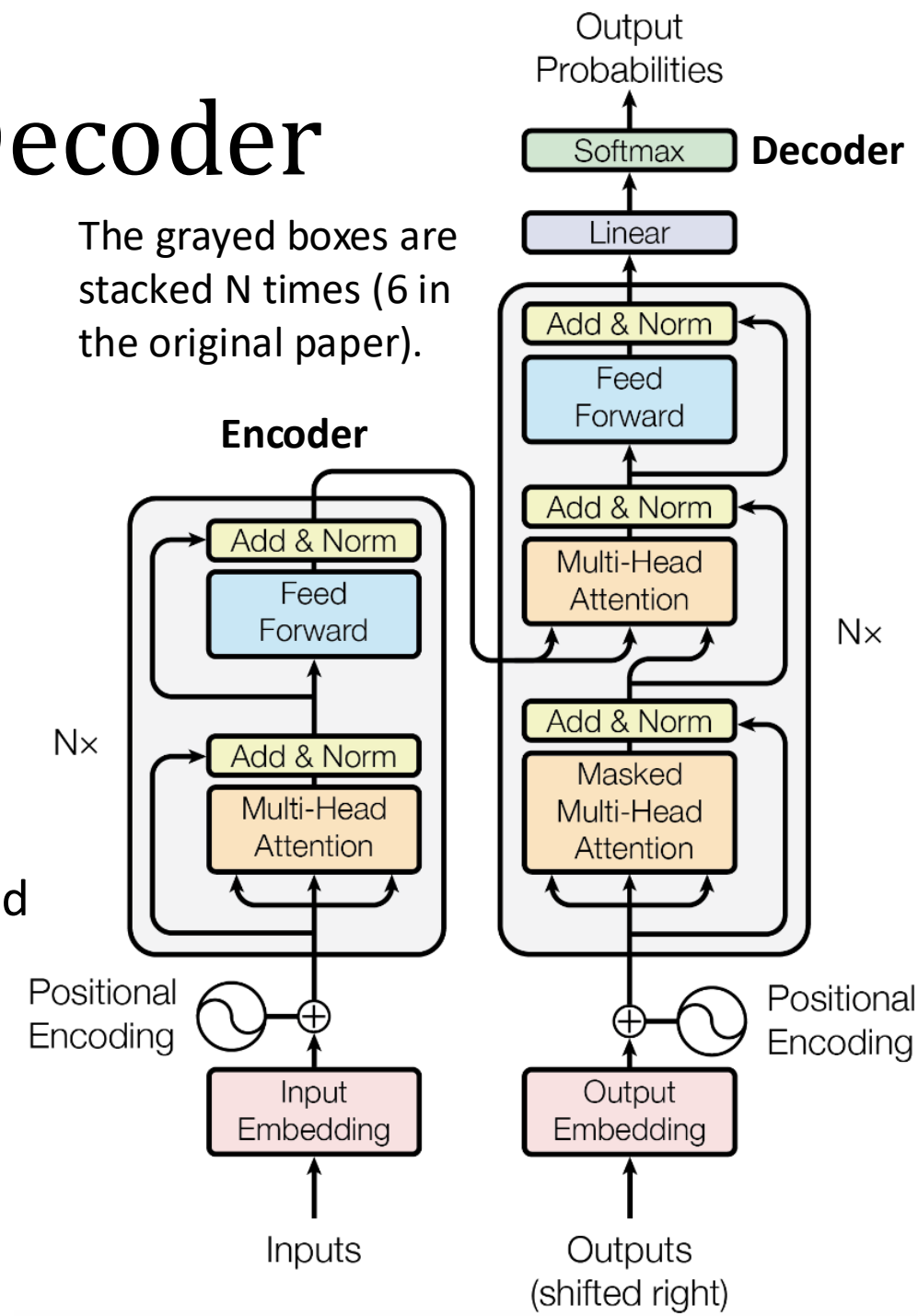
Full Encoder-Decoder Architecture

For when you want full bidirectional attention for computing representations of the input sequence,

AND you also want to generate an output sequence with masked attention contextualized by the input.

Example: Machine Translation.

The grayed boxes are stacked N times (6 in the original paper).

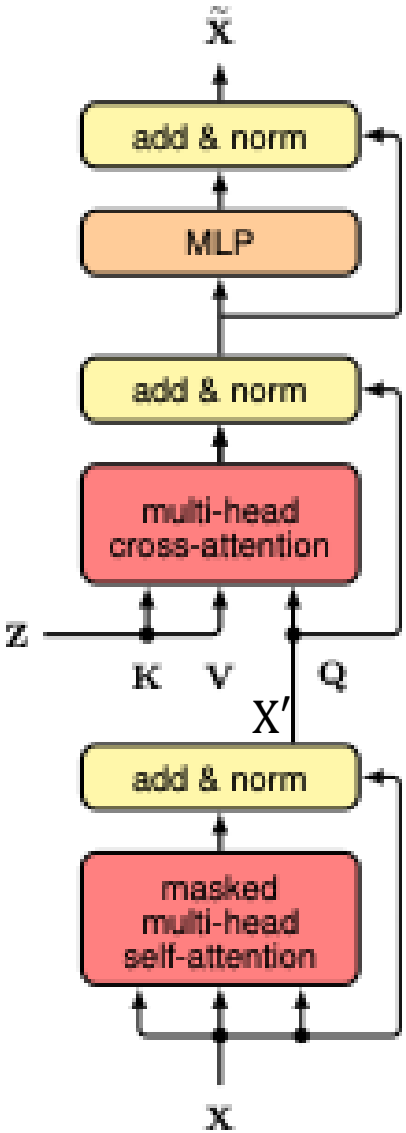


Cross-Attention Mechanism

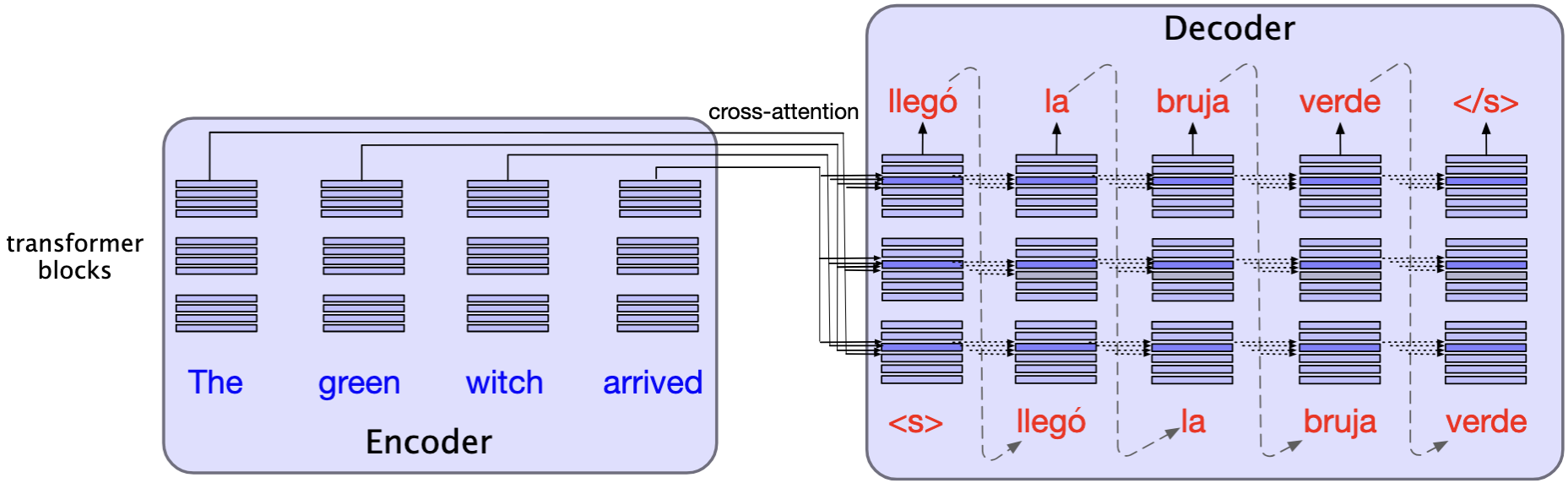
- Query matrix $\mathbf{Q} := \mathbf{X}'\mathbf{W}^{(q)}$
- Key matrix $\mathbf{K} := \mathbf{Z}\mathbf{W}^{(k)}$
- Value matrix $\mathbf{V} := \mathbf{Z}\mathbf{W}^{(v)}$

Where:

- $\mathbf{W}^{(q)}$, $\mathbf{W}^{(k)}$, and $\mathbf{W}^{(v)}$ coefficients belong to the cross-attention layer,
- $\mathbf{X}'$ is the output of the masked multi-head self-attention,
- $\mathbf{Z}$ is the last representation of the encoder stack,
- (omitting multi-head notation for simplicity)



Cross Attention in an Encoder-Decoder Transformer for Machine Translation



Note that the encoder's context is provided in cross attention at every block of the decoder.

Supervised Training and Aligned Text for Machine Translation

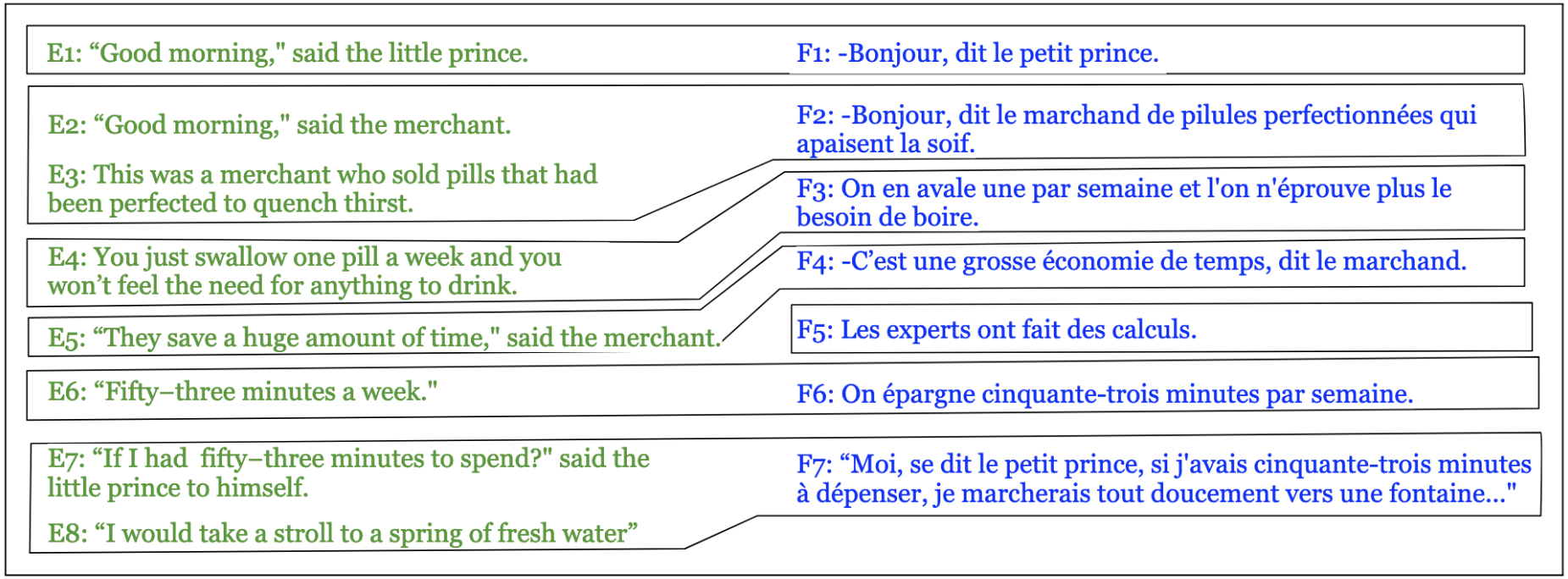


Figure 13.4 A sample alignment between sentences in English and French, with sentences extracted from Antoine de Saint-Exupery’s *Le Petit Prince* and a hypothetical translation. Sentence alignment takes sentences $e_1, \ldots, e_n$, and $f_1, \ldots, f_n$ and finds minimal sets of sentences that are translations of each other, including single sentence mappings like (e_1,f_1), (e_4,f_3), (e_5,f_4), (e_6,f_6) as well as 2-1 alignments (e_2/e_3,f_2), (e_7/e_8,f_7), and null alignments (f_5).